



Final Report

ICT 1209 - Web Technologies

**Bachelor of Information and Communication Technology
(BICT)**

Degree Programme

Department of Information and Communication Technology

Faculty of Technology

Rajarata University of Sri Lanka

Course: ICT 1209 - Web Technologies

Project Theme: Quiz/Trivia Game

Git Repository: <https://github.com/shehansandiv2003-hash/Quiz-verse.git>

Institution: Rajarata University of Sri Lanka, Department of ICT

Date: September 25, 2026

Team Members:

Name	Registration Number	Index Number
K.H.S.S.J. Wickramasooriya	ITT/2024/117	2802
W.D.D.M. Weerasinghe	ITT/2024/115	2800

Table of Contents

1. Project Overview	3
2. System Design	3
3. Implementation	5
4. Challenges and Solutions	6
5. Individual Contribution	6
6. Setup Instructions	7
7. References	7

Figure 1 – System architecture: browser, Apache/PHP, and the MySQL database	3
Figure 2 – Entity-Relationship diagram of the quiz_verse database.	4
Figure 3 – Page navigation and authentication flow.	5

1. Project Overview

Quiz-Verse is a dynamic, web-based trivia application designed to provide an engaging platform for testing knowledge across various subjects. The core purpose of the application is to replace static, hard-coded question sets with dynamically generated quizzes, ensuring a fresh and replayable experience for every session.

Target Audience:

- Students and self-learners reinforcing knowledge.
- Casual gamers and trivia enthusiasts.
- Teachers or trainers needing informal assessment tools.
- Friend groups looking for competitive, leaderboard-driven gameplay.

Key Features:

- **Dynamic Generation:** Configurable categories including Geography, History, Science, and ICT.
- **Instant Feedback & Timers:** A visual SVG countdown timer adds challenge, and immediate feedback is provided after each answer submission.
- **Score Tracking & Leaderboards:** Real-time progress tracking, with top scores permanently logged and displayed on a global "Top 10 Explorers" leaderboard.

2. System Design

The application follows a structured, standard web directory pattern and utilizes a relational database for state management.

Folder Structure:

The project conforms to the required file organization:

- **css/ and js/:** Contains custom stylesheets (style.css) and client-side logic (script.js).
- **includes/:** Houses reusable backend logic, including db.php for PDO/MySQLi connections and functions.php.
- **auth/:** Contains login.php, register.php, and logout.php for session management.

Root Files: Core application pages including index.php, quiz.php, dashboard.php, and contact.php.

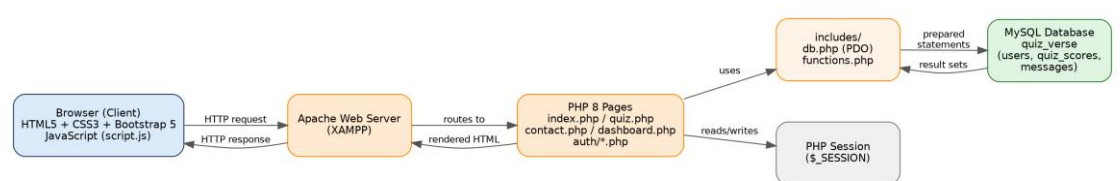


Figure 1 – System architecture: browser, Apache/PHP, and the MySQL database

Database Schema (quiz_verse.sql):

The database utilizes three primary tables to handle user data, contact inquiries, and gameplay statistics.

- **users Table:** Stores authenticated profiles with fields for id (AUTO_INCREMENT), username, email, password, and created_at timestamp.
- **quiz_scores Table:** Logs gameplay results with id, user_id, score, and played_at.
- **messages Table:** Captures contact form submissions with fields for id, name, email, message, and created_at.

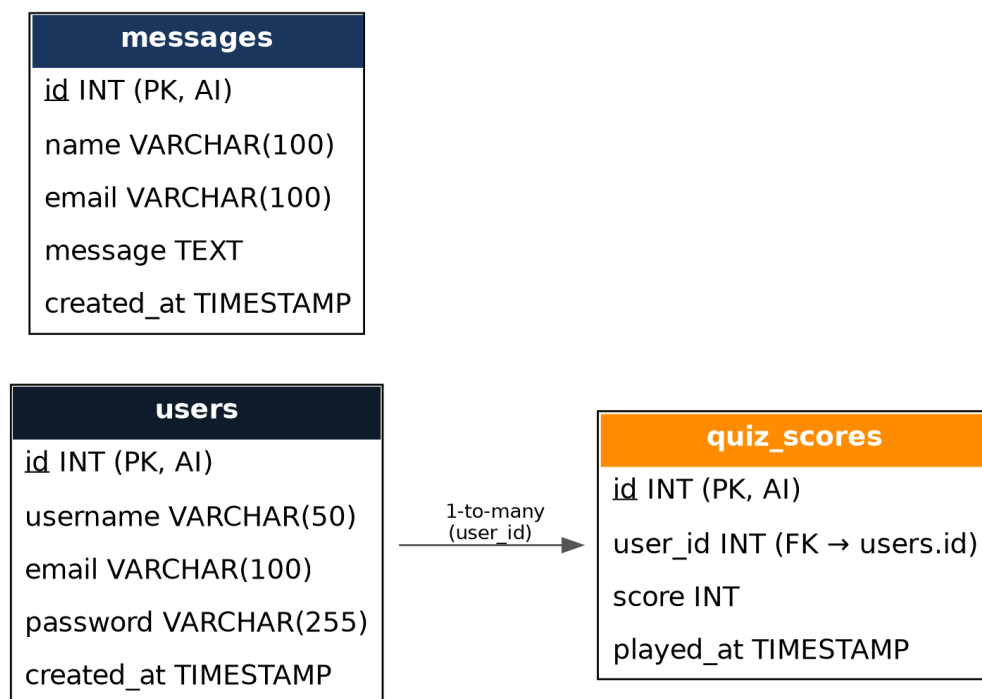


Figure 2 – Entity-Relationship diagram of the quiz_verse database.

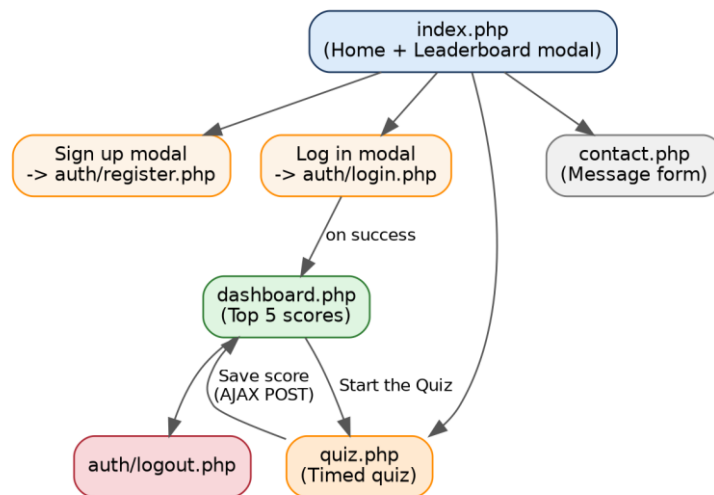


Figure 3 – Page navigation and authentication flow.

3. Implementation

Quiz-Verse is built on a designated technology stack combining a responsive frontend with a robust PHP backend.

- **Frontend (HTML5, CSS3, Bootstrap 5):** The UI features a dark, space-themed aesthetic. Bootstrap 5 handles structural components such as the responsive navigation bar, modals for login/signup, and accordion menus for the FAQ section.
- **Backend (PHP 8 & MySQL):** PHP scripts handle server-side processing, session management (`session_start()`), and secure database queries. Prepared statements are actively utilized to prevent SQL injection vulnerabilities.

For example, `dashboard.php` starts the session and redirects anonymous visitors before any dashboard content is rendered:

```

session_start();
require_once 'includes/functions.php';
require_once 'includes/db.php';

if (!isLoggedIn()) {
    header("Location: login.php");
    exit();
}

```

- **Data Integration:** The leaderboard dynamically renders the top 10 users by executing an SQL JOIN between the `quiz_scores` and `users` tables, ordering the results by score in descending order. Score saving is achieved asynchronously via POST requests to `quiz.php` checking for the `save_score` action.

Leaderboard query (`index.php` / `dashboard.php`):

```

$sql = "SELECT users.username, quiz_scores.score
        FROM quiz_scores
        JOIN users ON quiz_scores.user_id = users.id
        ORDER BY quiz_scores.score DESC
        LIMIT 5";
$stmt = $pdo->query($sql);
$top_scores = $stmt->fetchAll();

```

Score-saving endpoint (quiz.php), intercepted before any HTML is sent:

```
if (isset($_POST['action']) && $_POST['action'] === 'save_score') {
    if (isset($_SESSION['user_id']) && isset($_POST['score'])) {
        $user_id = $_SESSION['user_id'];
        $score = $_POST['score'];
        $stmt = $conn->prepare(
            "INSERT INTO quiz_scores (user_id, score) VALUES (?, ?)"
        );
        $stmt->bind_param("ii", $user_id, $score);
        $stmt->execute();
        echo "Score saved successfully!";
    } else {
        echo "Error: User not logged in.";
    }
    exit();
}
```

Contact form insert (contact.php), using a PDO prepared statement:

```
$name = sanitizeInput($_POST['name']);
$email = sanitizeInput($_POST['email']);
$message = sanitizeInput($_POST['message']);

$sql = "INSERT INTO messages (name, email, message) VALUES (?, ?, ?)";
$stmt = $pdo->prepare($sql);
$stmt->execute([$name, $email, $message]);
```

4. Challenges and Solutions

- **State Management & UI Toggles:** Managing the navigation bar to correctly display "Dashboard" and "Log Out" for authenticated users versus "Log in" and "Sign up" for guests required careful session handling. This was resolved by implementing `isset($_SESSION['user_id'])` conditional checks directly within the Bootstrap navbar components.
- **Score Persistence:** Saving scores dynamically without interrupting the frontend quiz flow presented a challenge. This was addressed by capturing the POST request at the top of `quiz.php`, executing the INSERT via a prepared MySQLi statement, and immediately exiting the script to return a success message.

5. Individual Contribution

- K.H.S.S.J. Wickramasooriya (Index: 2802): Led the frontend UI design, implemented shared portions of the frontend layout and structure, and developed the backend PHP/MySQL integrations for user authentication, database management, and dynamic score tracking.
- W.D.D.M. Weerasinghe (Index: 2800): Authored the complete initial project proposal and collaborated on building various parts of the frontend HTML, CSS, and interactive JavaScript components.

6. Setup Instructions

To deploy the application locally in a development environment:

1. Install and launch XAMPP or WAMP and start both the Apache and MySQL modules.
2. Navigate to phpMyAdmin and create a new local database named quiz_verse.
3. Import the provided quiz_verse.sql file to automatically generate the required tables.
4. Extract the project files and place them into your local web server directory (e.g., htdocs for XAMPP) inside a folder named web technology.
5. Access the application via a web browser at [http://localhost/web technology/](http://localhost/web%20technology/).

7. References

[1] Rajarata University of Sri Lanka, Department of ICT. *ICT 1209 - Web Technologies: Mini Project Guide | 2024 Batch*.

[2] Bootstrap Team. *Bootstrap 5 Documentation*

[3] Open Trivia Database API.